Spring Boot 3.X 개발 기본

inky4832@daum.net

3. Spring MVC

- Spring MVC 개요
- Spring MVC 아키텍처
- @Controller, @Service, @Repository
- Thymeleaf 개요
- 예외처리 및 유효성 검증

3.1 Spring MVC

Spring MVC

- 개념
  - 웹 어플리케이션의 구조적 설계 패턴 중 하나로서 역할별로 구분하는 아키텍처
  - 데이터처리, 화면처리, 제어 로직을 분리하는 것이 목적

- 구성요소별 역할

구성요소	설명	Spring 역할
Model	비즈니스 로직 및 데이터 처리	Service 및 Repository 계층과 연결 @Service, @Repository 사용
View	사용자에게 보여지는 화면 처리	Thymeleaf, JSP, XML, JSON 응답
Controller	요청(request)와 응답(response)제어	@Controller, @RestController

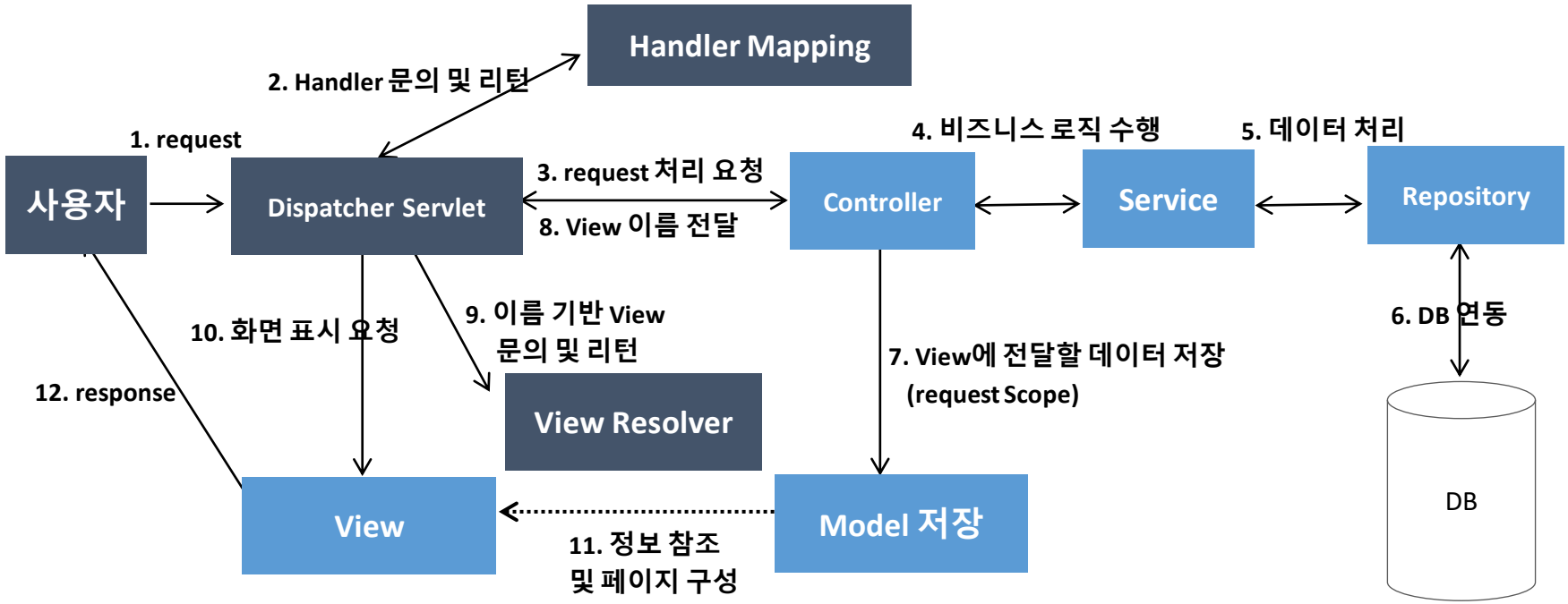
3.1 Spring MVC

Spring MVC 특징

구분	설명
Annotation 기반 개발	XML 설정 없이 어노테이션(ex. @Controller, @GetMapping 등)으로 구성
Front Controller 패턴	모든 요청은 DispatcherServlet 이 단일 진입점 역할
유연한 View 처리	Thymeleaf, JSP, XML, JSON 등 다양한 View 형식 지원 및 교체 용이
POJO 기반 구조	특별한 인터페이스 상속 없이 순수 Java 객체로 개발 가능
Spring IoC 통합	MVC 구조 내에서도 IoC/DI 로 객체 간 의존성 자동 관리
Exception 처리 내장	Controller 단위 또는 전역 예외 처리 지원 ex. @ExceptionHandler, @ControllerAdvice
RESTful 아키텍처 지원	REST API 방식으로 데이터를 JSON 형태로 응답
Spring Boot 자동 설정	spring-boot-starter-web 의존성으로 MVC 환경 자동 구성

3.1 Spring MVC

Spring MVC 요청 흐름



3.1 Spring MVC

주요 컴포넌트

구성 요소	설명
DispatcherServlet	요청을 받아서 Controller로 전달하고 결과를 반환하는 중앙 처리 기능
HandlerMapping	요청 URL과 매핑된 Controller 탐색
Controller	실제 요청을 처리하는 핵심 비즈니스 로직 호출 @Controller 또는 @RestController 사용
Service	비즈니스 로직 및 트랜잭션 처리 @Service , @Transactional 사용
Repository	DB 연동 및 데이터 처리 @Repository 사용
Model	Controller와 View 사이의 데이터 전달 객체
ViewResolver	논리적 View 이름을 실제 View 객체로 변환
View	사용자에게 출력되는 화면 (View Template)

3.1 Spring MVC

spring-boot-starter-web 의존성

- 개념
  - 웹 어플리케이션 개발에 필요한 모든 라이브러리를 한 번에 포함하는 Starter
  - implementation 'org.springframework.boot:spring-boot-starter-web'

- 주요 특징

특징	설명
생산성 향상	개발자가 웹 서버 설정 없이 바로 개발 가능
표준화	Spring MVC 구조를 자동 구성
단순화	Servlet 설정, DispatcherServlet 등록 자동 처리
내장 서버	Tomcat 자동 제공으로 별도 설치 불필요
확장성	REST API, 웹 MVC, JSON, 예외처리까지 모두 지원

3.1 Spring MVC

구성 라이브러리

포함 라이브러리	주요 기능	설명
spring-boot-starter	공통 설정	로깅, 자동 설정 등 공통 구성 포함
spring-boot-starter-json	JSON 처리	Jackson 기반 JSON 직렬화/역직렬화 직렬화: Java객체 -> JSON 변환 역직렬화: JSON -> Java 객체 변환
spring-web	서블릿 기반 웹 기능	DispatcherServlet, Controller, Filter 등
spring-webmvc	MVC 아키텍처	Model, View, Controller 패턴 지원
spring-boot-starter-tomcat	내장 서버	Tomcat 내장 웹 서버 자동 실행, **기본: 8080**
spring-boot-starter-logging	로깅	SLF4J + Logback 로깅 지원

3.1 Spring MVC

DispatcherServlet

- 개념
  - 모든 요청을 중앙에서 처리하는 프론트 컨트롤러 역할
  - 요청을 적절한 Controller로 위임하고 응답을 View로 전달

- 특징

항목	설명
단일 진입점	모든 요청이 DispatcherServlet을 거침
흐름 제어	요청 -> 처리 -> 응답 전 과정을 관리
확장성	다양한 View 기술과 연동 가능
자동 등록	Spring Boot 에서는 자동으로 DispatcherServlet 등록됨

3.1 Spring MVC

Controller

- 개념
  - 사용자의 요청을 받아 처리하는 핵심 컴포넌트
  - 요청 파라미터를 받고 비즈니스 로직(Service)을 호출한 뒤 결과를 Model에 담아 반환

- 주요 어노테이션

어노테이션	설명
@Controller, @RestController	Spring MVC의 Controller 클래스로 등록
@RequestMapping	URL 패턴과 Controller 메서드 매핑 정의
@GetMapping, @PostMapping	HTTP 메서드별 매핑 단축 어노테이션
@RquestParam, @ModelAttribute	파라미터 매핑 및 모델 바인딩
@ResponseBody, @RequestBody	View가 아닌 데이터(ex. JSON) 직접 반환 및 객체 매핑

3.1 Spring MVC

Controller 특징

- 요청 기반 매핑 구조
  - URL, HTTP Method(GET,POST)에 따라 처리 메서드가 구분됨
    ex. @GetMapping, @PostMapping
- 어노테이션 기반 설정
  - @Controller 또는 @RestController을 사용하여 Bean으로 등록
- POJO 기반
  - 일반 Java 클래스(Plain Old Java Object)로 작성 가능하며 Spring이 자동으로 관리
- 의존성 주입 지원
  - @Autowired 또는 생성자를 이용해 Service, Repository 등의 Bean을 주입 받아 사용

3.1 Spring MVC

@RequestMapping

- 개념
  - 사용자가 요청한 URL 정보를 이용하여 Controller의 실제 처리 메서드와 매핑하는 역할
- 특징
  - 가장 범용적으로 사용되는 방법
  - @Controller / @RestController의 클래스 수준과 메서드 수준에 경로를 선언
  - HTTP 메서드 특화된 어노테이션 사용 권장

    ex. **@GetMapping, @PostMapping, @PutMapping, @DeleteMapping**

3.1 Spring MVC

기본 사용 예

요청 경로	설명
@RequestMapping(value="/hello")	/hello 경로로 들어오는 요청을 처리. value 속성은 생략가능 ex. http://localhost:8080/hello
@RequestMapping(value="/main*")	* 는 한 단계의 와일드 카드로서 /main으로 시작하는 모든 경로 처리 가능 ex. /main, /mainA, /mainTest
@RequestMapping("/admin/**/user")	** 는 여러 경로 계층 와일드 카드로서 /admin으로 시작해서 /user로 끝나는 모든 경로 처리 가능 ex. /amin/user, /admin/a/user, /admin/a/b/user
@RequestMapping(value="/hello", method=RequestMethod.GET)	GET 방식의 요청 @GetMapping("/hello") 와 동일
@RequestMapping(value="/hello", method=RequestMethod.POST)	POST 방식의 요청 @PostMapping("/hello") 와 동일

3.1 Spring MVC

@RequestParam

- 개념
  HTTP 요청의 파라미터 값을 Controller 메서드의 매개변수로 매핑
  필수 옵션이기 때문에 파라미터가 누락되면 400 Bad Request 에러 발생

속성	타입	기본값	설명
value	String	파라미터 명과 동일	요청 파라미터 이름 지정 ex. @RequestParam(value="id")
required	Boolean	true	필수 여부 지정 ex. @RequestParam(required=false)
defaultValue	String	없음	파라미터가 없을 때 기본값 지정 ex. @RequestParam(required=false, defaultValue="기본값")

3.1 Spring MVC

@RequestParam 예시

```
@Controller
public class HelloController {

   // GET   /hello 요청
   @GetMapping ("/hello")
   public String showHelloPage(Model  model) {
      model.addAttribute("message",  "Spring MVC 구조 실습");
      return "hello";  // 논리적인 View 이름 반환
   }

   // GET /user?id=abc123 요청
   @GetMapping("/user")
   @ResponseBody
   public String getUser(@RequestParam("id") String userId) {
      return "User ID: " + userId;  // 본문으로 반환
   }
```

```
@GetMapping("/page")
@ResponseBody
public String page(@RequestParam(required = false) Integer size) {
   return "페이지 크기: " + size;
}

@GetMapping("/search")
@ResponseBody
public String search(
      @RequestParam String keyword,
      @RequestParam(defaultValue = "1") int page) {
   return "검색어: " + keyword + ", 페이지: " + page;
}

@GetMapping("/params")
@ResponseBody
public String params(@RequestParam Map<String, String> paramMap) {
   return "요청 파라미터: " + paramMap;
}
```

3.1 Spring MVC

@PathVariable

- 개념
  - 요청 URL 경로 자체에 포함된 값(path variable)을 Controller 메서드의 매개변수로 매핑
    RESTful 한 API 설계에 자주 사용됨

속성	타입	기본값	설명
value	String	파라미터 명과 동일	요청 파라미터 이름 지정 ex. @PathVariable(value="id")
required	Boolean	true	필수 여부 지정 ex. @PathVariable (required=false)
name	String	파라미터 명과 동일	value 와 동일 (가독성 용도)

3.1 Spring MVC

@PathVariable 예시

```
@Controller
public class HelloController {

   // GET   /hello 요청
   @GetMapping ("/hello")
   public String showHelloPage(Model model) {
      model.addAttribute("message", "Spring MVC 구조 실습");
      return "hello"; // 논리적인 View 이름 반환
   }

   // GET /item/10 요청
 @GetMapping("/item/{id}")
@ResponseBody
   public String getItem(@PathVariable("id") int id, Model model) {
      model.addAttribute("message", "상품번호: " + id);
      return "itemDetail";
   }
```

```
@GetMapping({"/user/{id}", "/user"})
@ResponseBody
public String getUser(@PathVariable(required = false) Integer id) {
   if (id == null) {
      return "전체 사용자 목록";
   }
   return "사용자 ID: " + id;
}
```

3.1 Spring MVC

@RequestParam vs @PathVariable

항목	@RequestParam	@PathVariable
입력방식	쿼리 파라미터 또는 폼 데이터	경로 변수
URL 형태	/product/detail?id=5	/products/5
Spring Mapping	@GetMapping("/product/detail") @RequestParam(“id”)	@GetMapping("/products/{id}") @PathVariable(“id”)
패턴	전통적 MVC 패턴	RESTful API

3.1 Spring MVC

Model

- 개념
  - Spring MVC 아키텍처에서 데이터를 저장하고 전달하는 객체
  - Controller가 비즈니스 로직을 수행한 결과를 View로 전달할 때 중간 매개체 역할
  - Model, ModelMap, ModelAndView, @ModelAttribute 등 사용

- 특징

항목	설명
Spring이 자동 관리	요청마다 새로운 Model 객체를 생성 관리
데이터 전달	Controller 가 View에 데이터를 전달
key/value 구조	model.addAttribute(key, value) 형태
스코프 관리	request 범위 내에서 유효
단순 구조	복잡한 DTO 없이도 간단히 데이터 전달 가능

3.1 Spring MVC

Model 예시

```
@GetMapping("/info")
public String showInfo(Model model) {
   model.addAttribute("title", "Spring MVC");
   model.addAttribute("version", "3.x");
   return "info";    // View 정보
}

@GetMapping("/info")
public ModelAndView showInfo() {
   ModelAndView mav = new ModelAndView();
   mav.addObject("title", "Spring MVC");
   mav.addObject("version", "3.x");
   mav.setViewName("info");     // View 정보
   return mav;
}
```

```
@GetMapping("/login")
public String login(@ModelAttribute("login") LoginDTO dto) {
    //모델 저장
    dto.setUserid("inky4832");
    dto.setPasswd("1234");
    return "success";
}
```

3.1 Spring MVC

Service

- 개념
  - Controller가 요청을 전달하면 Service는 필요한 데이터 처리/검증/계산 등을 수행
  - 비즈니스 로직을 수행하는 계층으로서 Repository를 통해서 DB에 접근

- 주요 역할

구분	설명
비즈니스 로직 처리	ex. 주문 시 재고 차감, 가격 계산, 유효성 검증
트랜잭션 처리	데이터 변경 시 오류가 생기면 롤백 처리 **(@Transactional** 사용)
Repository 호출	DB 저장, 수정, 삭제, 조회 등을 Repository에 위임
데이터 가공	DTO 변환, 계산, 검증 등을 수행
Controller와 Repository 사이의 중간 계층	역할 분리로 유지보수 용이

3.1 Spring MVC

Service

- 장점
  - 코드 재사용
  - 유지보수성
  - 안정성
  - 테스트 용이

- 주요 어노테이션

어노테이션	설명
@Service	해당 클래스가 비즈니스 로직을 담당하는 Service 계층임을 명시
@Transactional	메서드 실행 중 오류가 발생하면 모든 변경사항을 되돌림 (rollback 처리)

3.1 Spring MVC

Service 예시

```
@Service // 서비스 계층임을 명시
public class ProductService {

    private final ProductRepository productRepository;

    // 생성자 주입
    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    // 전체 상품 조회
    public List<Product> getAllProducts() {
        return productRepository.findAll();
    }

    // 상품 등록
    @Transactional
    public void addProduct(Product product) {
        if (product.getPrice() < 0) {
            throw new IllegalArgumentException("가격은 0 이상이어야 합니다.");
        }
        productRepository.save(product);
    }

    // 상품 삭제
    @Transactional
    public void deleteProduct(Long id) {
        productRepository.deleteById(id);
    }
}
```

3.1 Spring MVC

Repository

- 개념
  - 데이터를 영구적으로 저장하고 조회하는 계층
  - 어플리케이션에서 사용하는 도메인 객체(Entity, VO, DTO 등)와 실제 데이터 저장소 (DB, 파일, 외부 API 등) 사이의 추상화 계층 (Abstract Layer) 역할

- 역할

구분	설명
데이터 접근 (Data Access)	DB, 파일, 외부 API 등 다양한 저장소에 데이터를 읽고 쓰는 책임
추상화 (Abstraction)	구체적인 데이터 접근 기술(JDBC, MyBatis, JPA 등)을 숨기고 일관된 인터페이스 제공
Entity 관리	비즈니스 로직에서 사용하는 객체를 데이터 저장 구조에 맞게 변환

3.1 Spring MVC

Repository

- 계층 필요성

항목	설명
관심사 분리	Service는 비즈니스 로직에 집중하고 Repository는 데이터 접근에 집중
유지보수 용이성	데이터 접근 방식을 변경해도 Service 로직 변경 최소화
테스트 용이성	Mock 객체로 Repository 단위 테스트 가능
기술 교체 유연성	JPA -> MyBatis -> MongoDB 교체 시 Repository 인터페이스만 수정하면 됨

3.1 Spring MVC

Repository

- 주요 구성 요소

구성 요소	설명
Entity / Domain 객체	실제 저장할 데이터의 구조 정의 ex. Product, User 등
Repository Interface	데이터 접근 기능을 정의하는 추상 인터페이스
Repository 구현체	JDBC, MyBatis, JPA 등 실제 접근 기술을 사용해 구현. @Repository 이용

3.1 Spring MVC

Repository 예시

```
// 도메인 객체
public class Product {
    private Long id;
    private String name;
    private int price;

}
```

```
// Repository 인터페이스
public interface ProductRepository {
    Product save(Product product);            // 등록/수정
    Optional<Product> findById(Long id);      // 단일 조회
    List<Product> findAll();                  // 전체 조회
    void deleteById(Long id);                 // 삭제
}
```

```
// JDBC 구현체
@Repository
public class JdbcProductRepository implements ProductRepository {

    private final JdbcTemplate jdbcTemplate;

    public JdbcProductRepository(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    @Override
    public Product save(Product product) {
        String sql = "INSERT INTO product(name, price) VALUES(?, ?)";
        jdbcTemplate.update(sql, product.getName(), product.getPrice());
        return product;
    }
    @Override
    public void deleteById(Long id) {
        jdbcTemplate.update("DELETE FROM product WHERE id = ?", id);
    }
}
```

3.1 Spring MVC

Repository 예시

```
 // MyBatis 구현체
@Mapper
public interface MyBatisProductRepository
                        extends ProductRepository {

   @Override
   Product save(Product product);

   @Override
   Optional<Product> findById(Long id);

   @Override
   List<Product> findAll();

   @Override
   void deleteById(Long id);
}
```

```
 // Memory 구현체
@Repository
public class MemoryProductRepository implements ProductRepository {

   private final Map<Long, Product> store = new HashMap<>();
   private final AtomicLong sequence = new AtomicLong(0);

   @Override
   public Product save(Product product) {
      long id = sequence.incrementAndGet();
      product = new Product(id, product.getName(), product.getPrice());
      store.put(id, product);
      return product;
   }
@Override
   public List<Product> findAll() {
      return new ArrayList<>(store.values());
   }
   @Override
   public void deleteById(Long id) {
      store.remove(id);
   }
}
```

3.1 Spring MVC

핵심기능 비교

구분	Controller	Service	Repository
주요 역할	요청/응답 처리	로직 수행	데이터 저장/조회
Spring 어노테이션	**@Controller**	**@Service**	**@Repository**
트랜잭션 관리	X	O	X
핵심 키워드	요청 흐름	비즈니스	데이터

3.1 Spring MVC

View

- 개념
  - Spring MVC 패턴에서 클라이언트에게 응답을 표현하는 계층
  - Controller가 처리한 결과(Model 데이터)를 받아 사용자가 볼 수 있는 형태로 출력
    ex. HTML, XML, JSON 등

- 핵심 역할

구분	역할
응답 생성 (Presentation)	• Controller에서 전달된 Model 데이터를 화면(HTML) 또는 JSON데이터로 변환
표현 기술 독립성	• View 종류(JSP, Thymeleaf, JSON 등)는 자유롭게 교체 가능 • Controller는 View 기술에 독립적
Model 데이터 접근	• View는 Model 객체를 통해 데이터에 접근
표현 로직 처리	• 단순한 데이터 표시 (비즈니스 로직은 없음)

3.1 Spring MVC

View 형태

View 유형	설명
HTML View	템플릿 엔진 (JSP, Thymeleaf, FreeMarker 등)을 이용해 화면 생성
JSON View	REST API 응답 시 객체를 JSON 형태로 반환
파일 기반 View	PDF, Excel 등의 파일 형식으로 응답 생성
Redirect View	다른 URL로 재요청을 보내는 리다이렉션 수행 ex. redirect:/path
Forward View	동일 요청 내에서 다른 컨트롤러로 제어 이동 ex. forward:/path

3.1 Spring MVC

View 이름 반환 방식

- 개념
  - Controller는 View 객체를 직접 반환하지 않음
  - View의 논리적 이름(String)만 반환
  - 논리적 이름은 ViewResolver가 실제 View 객체로 변환시킴

    ex. ThymeleafView, JstlView, MappingJackson2JsonView

```
ex.
@Controller
public class HomeController {

   @GetMapping("/home")
   public String home(Model model) {
      model.addAttribute("message", "환영합니다!");
      return "home";   // "home"은 논리적 View 이름
   }
}
```

3.1 Spring MVC

ViewResolver

- 개념
  - Controller에서 반환된 View의 논리적 이름을 실제 View 객체로 변환

- 핵심 유형

유형	설명
Internal Resource View Resolver	JSP 같은 서버 사이즈 HTML 템플릿을 찾을 때 사용
Thymeleaf View Resolver	Thymeleaf 템플릿을 찾을 때 사용
Free Marker View Resolver	FreeMarker 템플릿을 찾을 때 사용
Content Negotiating View Resolver	요청 헤더(Content-Type)에 따라 HTML,JSON 등 View 자동 선택

3.1 Spring MVC

ViewResolver 동작 흐름

Controller 에서 View 이름 반환

```
@GetMapping("/info")
public String info(Model model) {
    model.addAttribute("title", "정보 페이지");
    return "info";  // View 이름만 반환
}
```



ViewResolver가 처리

입력	처리 결과
View 이름 “info”	실제 파일 경로 변환 JSP 경우: /WEB-INF/views/info.jsp Thymeleaf 경우: templates/info.html
Model 데이터	{ “title”: “정보 페이지“}
최종 View	Model 데이터를 표현한 HTML 문서



최종 결과

클라이언트는 HTML, JSON 등 출력

ViewResolver가 내부에서 어떤 View기술을 사용했는지 완전히 숨김

3.1 Spring MVC

핵심기능 비교

항목	View	ViewResolver
정의	사용자에게 보여줄 응답을 생성하는 객체	View 이름을 실제 View 객체로 변환
역할	Model 데이터 표시	View 선택 및 연결
Controller 관계	Controller가 View이름을 반환하면 실행됨	Controller와 View 사이 조정
표현 기술 의존	없음 (HTML, JSON, PDF 등 모두 가능)	없음 (등록된 Resolver가 선택)
Spring 내 위치	MVC 의 V 역할 (출력 담당)	MVC의 V 조정자 역할

3.1 Spring MVC

Thymeleaf

- 개념
  - HTML 파일 내에서 동적으로 데이터를 표시하기 위한 서버사이드 템플릿 엔진
  - Controller에서 전달받은 데이터를 HTML 페이지에 랜더링
  - implementation 'org.springframework.boot:spring-boot-starter-thymeleaf'

- 특징

항목	설명
자연 템플릿	브라우저에서도 정상적으로 열리는 HTML 구조 유지
Spring MVC와 통합	Model 객체의 데이터를 HTML로 바로 전달 가능
표현식(Expression)	${변수} 문법으로 데이터 출력
조건/반복문 지원	th:if, th:each 사용

3.1 Spring MVC

Thymeleaf 기본 구조

```
src/
    main/
        java/
            com/example/demo
                DemoApplication.java

        resources/
            static/                 # JS, CSS, 이미지
            templates/              # Thymeleaf  HTML 파일 위치
                index.html
            application.yml
```

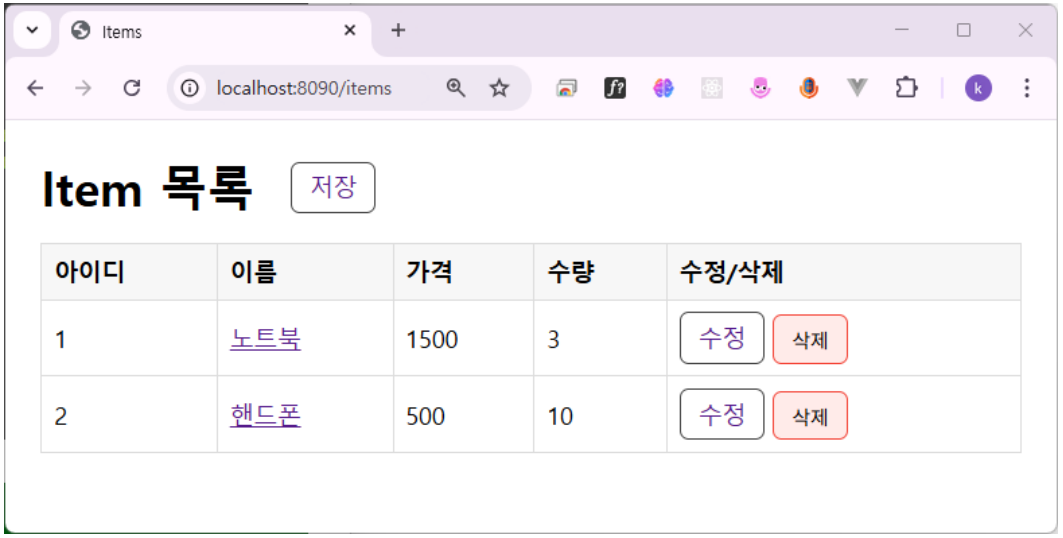
3.1 Spring MVC

Thymeleaf 기본 문법

형태	설명
${...}	변수 출력 ex. ${user.name}
*{...}	th:object 로 지정된 객체 선택 ex. *{email}
@{...}	정적/동적 링크 ex. @{/css/app.css}, @{/user/{id}(id=${u.id})}
th:text	텍스트 출력 ex. fallback
th:if, th:unless	조건 ex. <p th:if="${age >= 20}">성인</p><p th:unless="${age >= 20}">미성년자
th:switch, th:case	분기 ex. 관리자 일반 사용자 기타
th:each	반복 ex. <li th:each="item, stat : ${items}"><span th:text="${stat.index}">0: 이름
th:속성	속성 제어 ex. <img th:src="@{/images/logo.png}" alt="logo"> <a th:href="@{/users/{id}(id=${u.id})}">상세<input type="checkbox" th:checked="${u.active}">
th:action, th:object	객체 바인딩(Form) ex. <form th:action="@{/users}" th:object="${user}" method="post"> <input th:field="*{name}"><input th:field="*{email}"><button type="submit">저장</button></form>

3.1 Spring MVC

실습(MVC)




```
// build.gradle
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'org.springframework.boot:spring-boot-starter-thymeleaf'
    developmentOnly 'org.springframework.boot:spring-boot-devtools'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}
```

3.2 예외 처리와 검증

학습목표

- 예외처리 개요
- @ExceptionHandler
- @ControllerAdvice
- 검증(Validation) 개요
- 주요 검증 어노테이션
- @Validated 이용한 검증 실습

3.2 예외 처리와 검증

예외처리

- 개념
  - 예외(Exception)는 프로그램 실행 중에 발생하는 비정상적인 상황 의미
  - Spring MVC에서는 예외 발생 시 사용자에게 친절한 오류 화면 및 메시지를 제공하기 위한 예외 처리 메커니즘(Exception Handling Mechanism)을 제공
  - Spring에서는 대부분이 RuntimeException 기반 예외를 사용

- 특징

항목	설명
계층적 처리	Controller 단위, 전역 단위 등 다양한 범위에서 처리 가능
유지보수 용이	공통 예외 처리 로직을 별도 클래스로 분리 가능
일관된 응답	오류 페이지, 로그, 메시지를 일관된 방식으로 관리
어노테이션 기반	@ExceptionHandler, @ControllerAdvice 지원

3.2 예외 처리와 검증

@ExceptionHandler

- 개념
  - Spring MVC에서 Controller 내에서 발생한 예외를 처리하기 위해 사용
  - 특정 예외 타입을 지정하면, 해당 예외가 발생했을 때 이 메서드가 호출되어 응답을 제어 가능

- 특징

구분	설명
적용 범위	특정 Controller 클래스 내에서만 유효
어노테이션 위치	Controller 내부의 메서드 레벨
대상 예외 지정	어노테이션에 처리할 예외 클래스를 명시. 복수 지정 가능 ex. @ExceptionHandler(NullPointerException.class)
반환 타입	String, ModelAndView, void, ResponseEntity 등 다양

3.2 예외 처리와 검증

@ExceptionHandler

- 기본 문법

```
@ExceptionHandler(예외클래스명.class)
public 반환타입 메서드명(예외클래스명 e) {
   // 예외 처리 로직
   return 결과;
}

ex.
 @ExceptionHandler(UserNotFoundException.class)
   public String handleUserNotFound(UserNotFoundException  e, Model model) {
                  model.addAttribute("errorMessage",  e.getMessage());
                  return "error/customError";   // error 처리 화면
   }
```

3.2 예외 처리와 검증

@ExceptionHandler

- 반환 타입

타입	설명	예시
String	View 이름 반환	return “error/customError”;
Model And View	Model + View 동시 설정	return new ModelAndView(…);
Response Entity	JSON 응답 처리	return ResponseEntity.status(404).body(Map.of(“error”, e.getMessage()));

3.2 예외 처리와 검증

@ExceptionHandler 예시

```
@Controller
public class UserController {

    @GetMapping("/user/{id}")
    public String getUser(@PathVariable Long id, Model model) {
            if (id == 0) throw new UserNotFoundException("사용자를 찾을 수 없습니다.");
            return "user/detail";
    }

    @ExceptionHandler(UserNotFoundException.class)
    public String handleUserNotFound(UserNotFoundException e, Model model) {
            model.addAttribute("errorMessage", e.getMessage());
            return "error/customError";   // error 처리 화면
    }
}
```

3.2 예외 처리와 검증

@ControllerAdvice

- 개념
  - Spring MVC의 여러 Controller 에서 발생하는 예외를 처리하는 전역 예외처리
  - 공통으로 발생하는 예외를 하나의 클래스에서 관리

- 특징

구분	설명
적용 범위	전역(Global) , 모든 Controller 클래스 대상
어노테이션 위치	Controller 클래스 레벨
결합 어노테이션	@ExceptionHandler, @InitBinder, @ModelAttribute 와 같이 사용가능
스캔 대상	자동 스캔 가능 (@Component 특성 가짐)

3.2 예외 처리와 검증

@ControllerAdvice

- 기본 문법

```
@ControllerAdvice
public class GlobalExceptionHandler {

   @ExceptionHandler(예외클래스.class)
   public 반환타입 메서드명(예외클래스 e) {
      // 예외 처리 로직
      return 결과;
   }
}
```

3.2 예외 처리와 검증

@ControllerAdvice 예시

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(NullPointerException.class)
    public ModelAndView handleNullPointerException(NullPointerException ex) {
                ModelAndView mv = new ModelAndView("error/globalError");
                mv.addObject("errorMessage", "데이터가 존재하지 않습니다.");
                return mv;
    }

    @ExceptionHandler(Exception.class)
    public ModelAndView handleGeneralException(Exception ex) {
                ModelAndView mv = new ModelAndView("error/globalError");
                mv.addObject("errorMessage", "시스템 오류가 발생했습니다.");
                return mv;
    }
}
```

3.2 예외 처리와 검증

@ControllerAdvice vs @ExceptionHandler

항목	@ExceptionHandler	@ControllerAdvice
적용 범위	Controller 단위	전역 (모든 Controller 대상)
위치	Controller 내부 메서드 레벨	외부 공통 클래스의 클래스 레벨
주요 목적	개별 예외 처리	전역 공통 처리
확장성	낮음	높음
우선 순위	Controller 내부 Handler가 먼저 실행	이후 Global Handler가 실행

3.2 예외 처리와 검증

검증 (Validation)

- 개념
  - 사용자가 입력한 데이터가 시스템이 요구하는 형식, 범위, 조건 등을 만족하는지를 확인하는 과정
  - Controller 계층에서 주로 수행되고 DTO 수준에서 처리됨

- 주요 특징

특징	설명
데이터 무결성 보장	잘못된 형식의 데이터 입력을 방지
표준화 검증 API 제공	JSR-380 (Bean Validation 2.0, Jakarta Validation) 표준 기반 Hibernate Validator 가 기본 구현체로 사용
간결한 선언형 검증	어노테이션 기반 (@NotBlank 등)으로 직관적이고 코드 중복 방지
자동 통합	@Validated + Spring MVC 사용 시 자동 검증 지원

3.2 예외 처리와 검증

주요 검증 어노테이션

어노테이션	설명	적용 타입
@NotNull	null 값 불허	모든 타입
@NotBlank	공백 문자열 불허	String
@NotEmpty	빈 컬렉션/문자열 불허	String, List
@Size(min, max)	문자열/컬렉션 크기 제한	String, List
@Min, @Max	숫자 최소/최대값 제한	Number
@Email	이메일 형식 검증	String
@Pattern(regexp)	정규 표현식 패턴 검증	String
@Positive, @Negative	양수/음수 검증	Number
@Past, @Future	과거/미래 날짜 검증	Date, LocalDate

3.2 예외 처리와 검증

@Validated

- 개념
  - Spring Framework에서 제공하는 검증(Validation) 전용 어노테이션
  - 사용자가 전송한 데이터를 검증 객체(Validator)를 통해 유효한 값인지 검사
  - Bean Validation(JSR 380, Hibernate Validator)을 내부적으로 사용
  - 그룹(Group) 검증 기능 지원

- 의존성

```
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'org.springframework.boot:spring-boot-starter-thymeleaf'
    implementation 'org.springframework.boot:spring-boot-starter-validation'  // 검증 라이브러리
}
```

3.2 예외 처리와 검증

@Validated 예시

```
public class UserForm {

    @NotBlank(message = "이름은 필수 입력입니다.")
    private String name;

    @Size(min = 4, max = 12, message = "비밀번호는 4~12자 사이로 입력해야 합니다.")
    private String password;

    @FutureOrPresent(message = "현재 날짜보다 미래 날짜 입력")
    LocalDate targetDate;
    // getter / setter
}


@PostMapping("/register")
public String register(@Validated @ModelAttribute("userForm") UserForm form,
                       BindingResult result, Model model) {

    // 검증 결과 확인
        if (result.hasErrors()) {
            return "registerForm"; // 오류 발생 시 다시 입력 페이지로
        }
        …
        return "success";
}
```

3.2 예외 처리와 검증

@Validated 예시

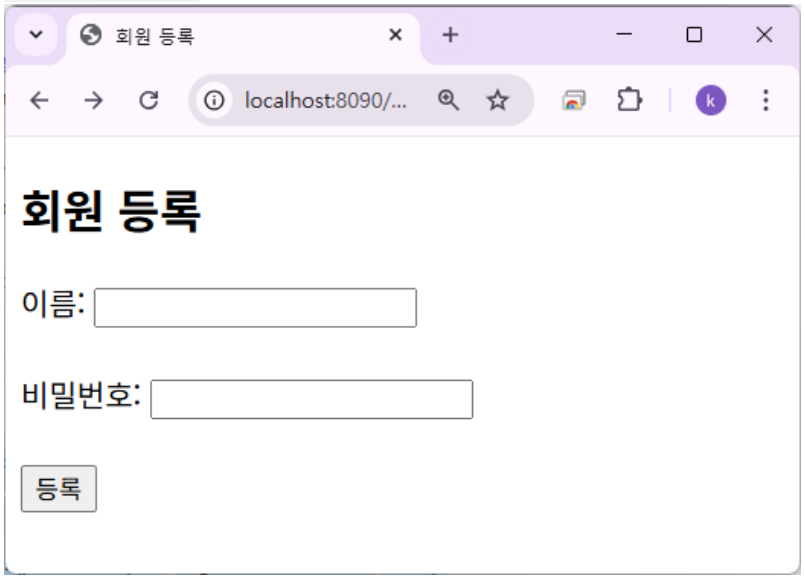
```
<!-- registerForm.html -->
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
 <head>
  <meta charset="UTF-8" />
  <title>회원 등록</title>
 </head>
 <body>
  <h2>회원 등록</h2>
  <form th:action="@{/user/register}" th:object="${userForm}" method="post">
   이름: <input type="text" th:field="*{name}" /><br />
   <span th:if="${#fields.hasErrors('name')}" th:errors="*{name}"></span><br />
   비밀번호: <input type="password" th:field="*{password}" /><br />
   <span th:if="${#fields.hasErrors('password')}" th:errors="*{password}"></span>
   <br />
   날짜: <input type="date" th:field="*{targetDate}" /><br />
   <span  th:if="${#fields.hasErrors('targetDate')}" th:errors="*{targetDate}"></span
   ><br />
   <button type="submit">등록</button>
  </form>
 </body>
</html>
```

```
<!-- success.html -->
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
 <head>
  <meta charset="UTF-8" />
  <title>등록 성공</title>
 </head>
 <body>
  <h2 th:text="${msg}"></h2>
  <a th:href="@{/user/register}">다시 등록하기</a>
 </body>
</html>
```

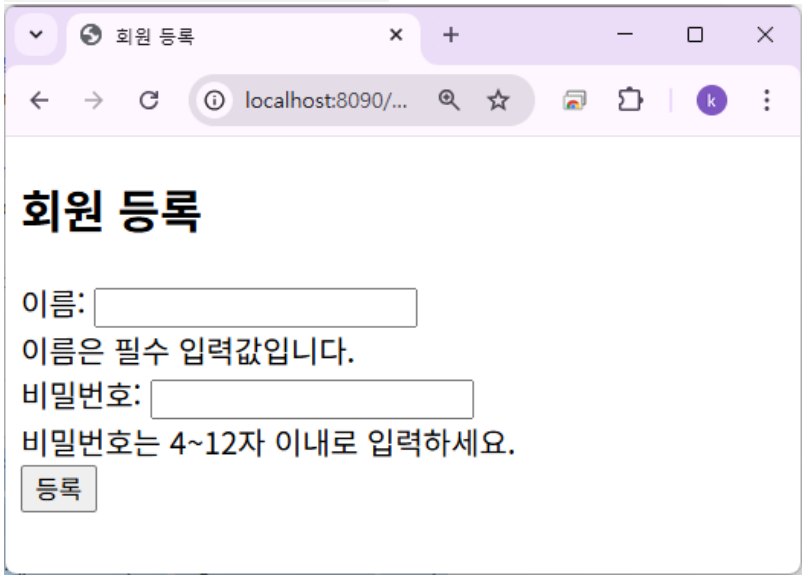
3.2 예외 처리와 검증

실행결과 (@Validated)

메인 화면



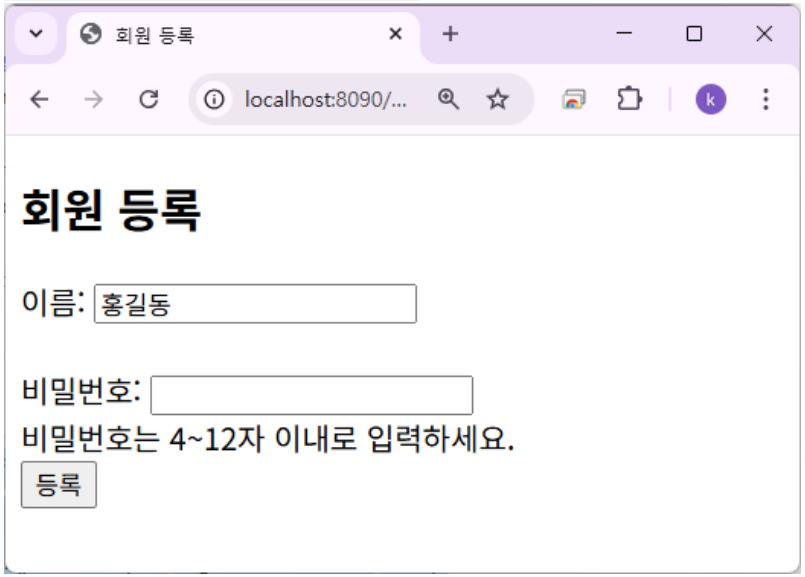
유효성 검증 실패 화면



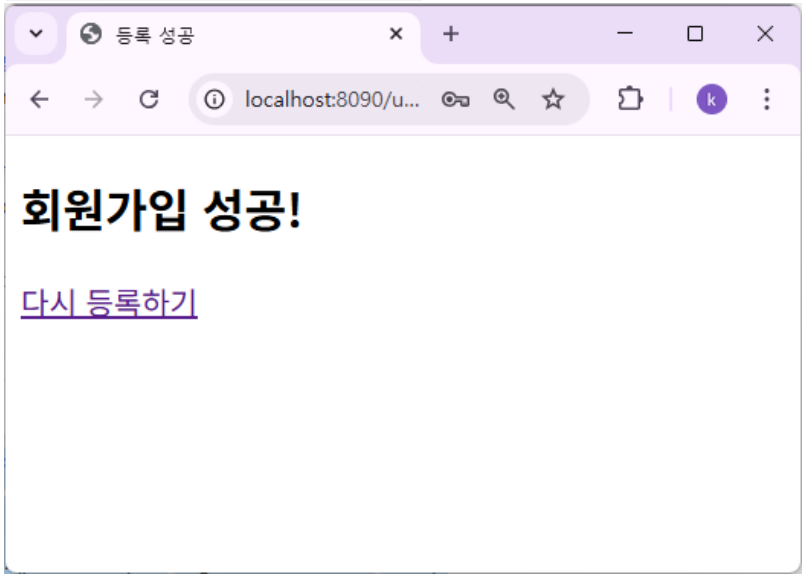
3.2 예외 처리와 검증

실행결과 (@Validated)

비밀번호 검증 실패 화면



유효성 검증 성공 화면



수고하셨습니다.